

Score Labspace

A learning experience for exploring and understanding the Score project and features.

Overview

👋 Welcome to the **Score Labspace**!

This is a learning experience for exploring and understanding the CNCF Sandbox project [Score](#) and its features.

You will be guided through instructions to run command lines with tools already pre-installed to better understand the concepts of Score and its features.

✅ Here is how this lab is structured:

- Introduction to Score
- The Score Specification
- Getting started with `score-compose`
- Getting started with `score-k8s`
- Advanced scenario with `score-compose`
- Advanced scenario with `score-k8s`
- Next steps

Let's Score it! 🎵

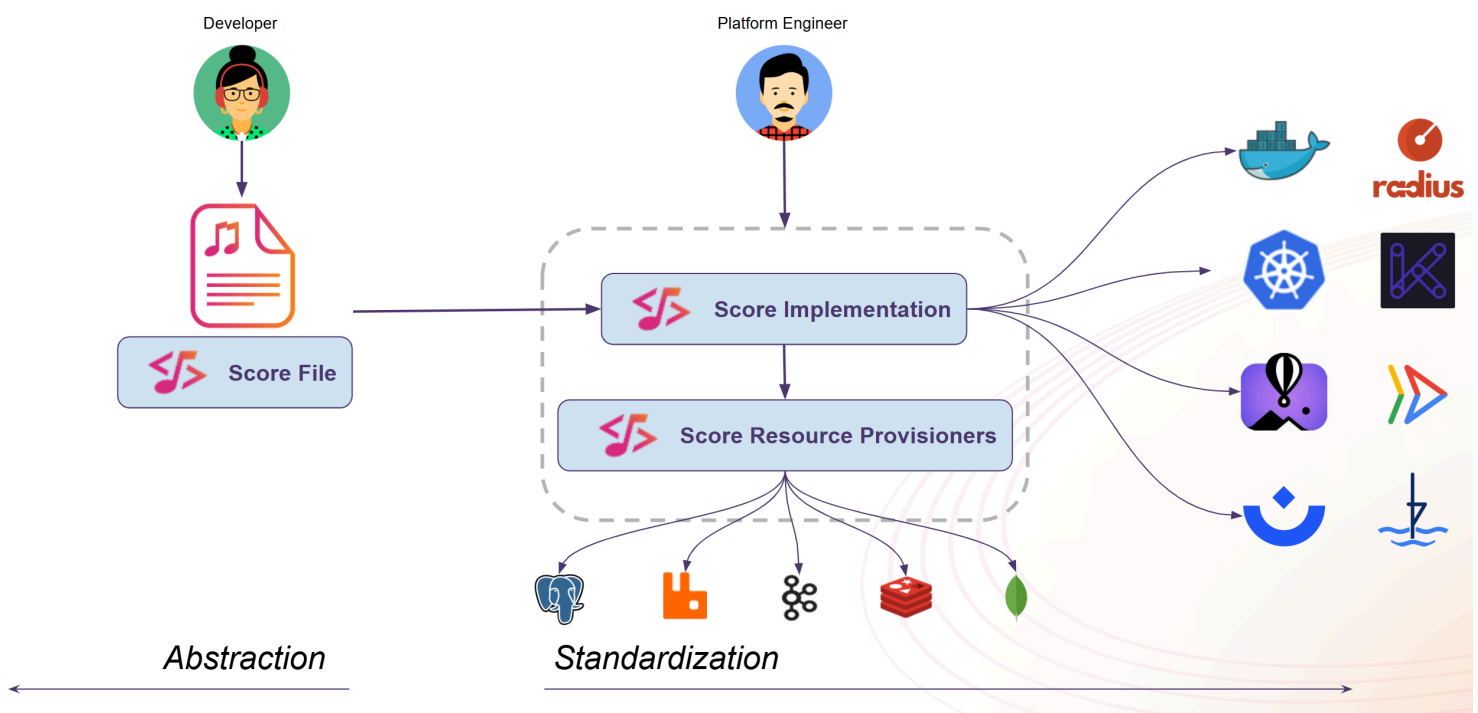
Enjoy! 🎉

What is Score?

Score is an open-source and a [CNCF Sandbox project](https://github.com/cncf/sandbox).

Score aims to reduce developer toil and cognitive load by enabling the definition of a single file that works across multiple platforms in a vendor-neutral way, eliminating the need for tooling-specific syntax from platforms such as Docker or Kubernetes for example.

On one hand ("**Abstraction**"), the Score spec allows **Developers** to define how they want to deploy their Workloads. And on the other hand ("**Standardization**"), **Platform Engineers** define well supported golden paths by using Score implementations supported for their platform(s) with associated resource provisioners.



Score specification

First and foremost, Score is a Specification.

The Score Specification is what Developers will use to describe in a YAML file what their workload needs. Eventually someone later will deploy it somewhere with a Score implementation.

Simple Score file

The simplest Score file that you can have is:

```
apiVersion: score.dev/v1b1
metadata:
  name: my-workload
containers:
  my-container:
    image: .
```

This is defining that the workload `my-workload` has one container `my-container`, and at this stage the container image name is not known (`.`) and will be supplied later in the CI/CD workflow.

Note: it's not a Kubernetes Custom Resource.

More complex Score file

Now here is a more realistic workload definition, a public service talking to a PostgreSQL database ([score.yaml](#)):

```
apiVersion: score.dev/v1b1
metadata:
  name: hello-world
  annotations:
    tags: "nodejs,http,website,javascript"
containers:
  hello-world:
    image: .
    variables:
      PORT: "3000"
      MESSAGE: "Hello, World!"
      DB_DATABASE: ${resources.db.name}
      DB_USER: ${resources.db.username}
      DB_PASSWORD: ${resources.db.password}
      DB_HOST: ${resources.db.host}
```

```
    DB_PORT: ${resources.db.port}
resources:
  db:
    type: postgres
  dns:
    type: dns
  route:
    type: route
    params:
      host: ${resources.dns.host}
      path: /
      port: 8080
service:
  ports:
    www:
      port: 8080
      targetPort: 3000
```

Resources

- [Score specification reference](#)

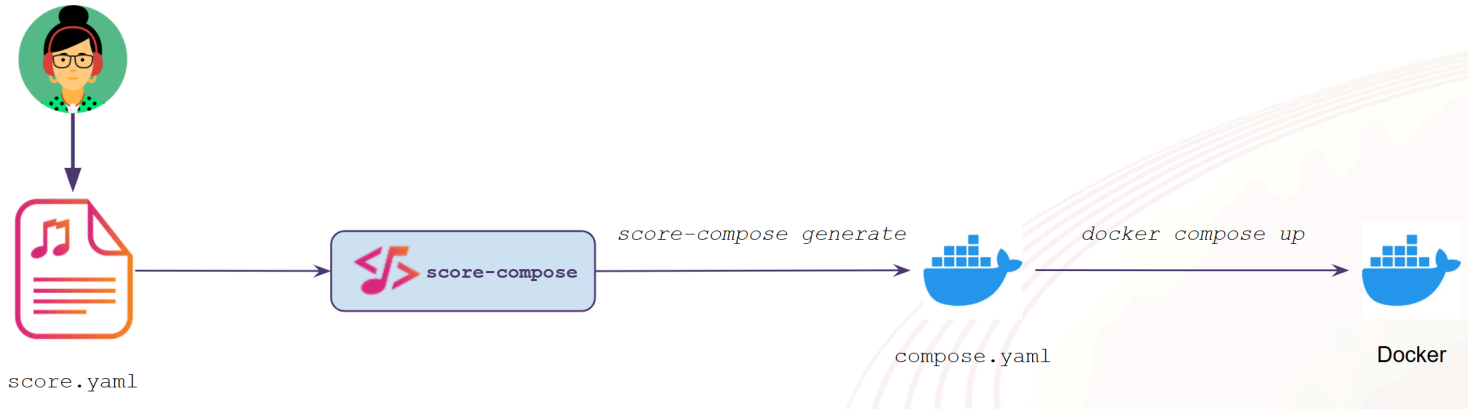
Getting started with **score-compose**

But from this **score.yaml** file we haven't deployed anything just yet.

Let's now deploy this workload locally with Docker Compose!

We'll use the **score-compose** implementation for that.

Developer



Initialize your current **score-compose** workspace, run the following command in your terminal:

```
score-compose init
```

*Note: The **init** command created the **.score-compose** directory to store a local state, read more about its purpose [there](#).*

Convert the **score.yaml** file into a deployable **compose.yaml**, run the following command in your terminal:

```
score-compose generate score.yaml \  
  --image scorespec/sample-score-app:latest
```

A new **compose.yaml** file has been generated, see [here](#).

All of this technical details abstracted by the **score-compose** implementation from the Developer.

Use Docker Compose to locally spin up our workload and its dependencies:

```
docker compose up -d
```

See the running containers:

```
docker ps
```

Test the deployed workload on [localhost:8080](#).

At this stage, we used two resource types: `postgres`, `dns` and `route`.

The available resource types and their inputs/outputs could be discovered like this:

```
score-compose provisioners list
```

We'll explore more about these resource types and provisioners later.

Resources

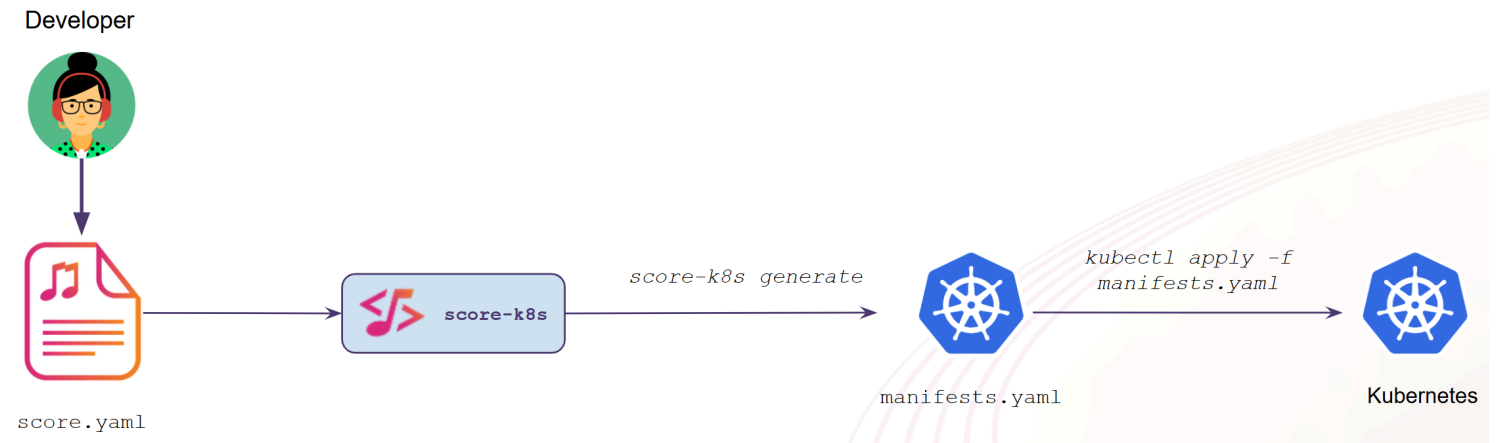
- [score-compose implementation](#)
- [Local state directory](#)

Getting started with **score-k8s**

From this exact same **score.yaml** file we now want to deploy it to Kubernetes.

Let's do it!

We'll use the **score-k8s** implementation for that.



Initialize your current **score-k8s** workspace, run the following command in your terminal:

```
score-k8s init
```

*Note: The **init** command created the **.score-k8s** directory to store a local state, read more about its purpose [there](#).*

Convert the **score.yaml** file into a deployable **manifests.yaml**, run the following command in your terminal:

```
score-k8s generate score.yaml \  
  --image scorespec/sample-score-app:latest
```

A new **manifests.yaml** file has been generated, see [here](#).

All of this technical details abstracted by the **score-k8s** implementation from the Developer.

Before deploying this **manifests.yaml** to Kubernetes, we need to cluster, create a local **kind** cluster by running this command:

```
sudo ./scripts/setup-kind-cluster.sh
```

Deploy these Kubernetes manifests to spin up our workload and its dependencies in Kubernetes cluster:

```
sudo kubectl apply -f manifests.yaml
```

See the running containers and associated resources:

```
sudo kubectl get all,statefulset,secret,httproute
```

Test the deployed workload on localhost:8080.

At this stage, we used two resource types: `postgres`, `dns` and `route`.

The available resource types and their inputs/outputs could be discovered like this:

```
score-k8s provisioners list
```

We'll explore more about these resource provisioners later.

Resources

- [score-k8s implementation](#)
- [Local state directory](#)

Advanced scenario with **score-compose**

The **score-compose** implementation CLI provides more options to enrich the default **compose.yaml** file generated in order to support more native Docker Compose features.

--build

Build the container image while deploying it via the **docker compose up --build** command, by using the **score-compose generate --build** feature:

```
score-compose generate score.yaml \  
  --build 'hello-world={"context":".", "tags":["hello-world"]}'
```

```
docker compose up --build -d
```

--override-property

Override any properties of the Score file imperatively:

```
score-compose generate score.yaml \  
  --image scorespec/sample-score-app:latest \  
  --override-property containers.hello-world.variables.MESSAGE="Hello, Compose!"
```

See the new **MESSAGE** value in the generated [compose.yaml](#) file.

--patch-templates

Override the default content and any fields of the **compose.yaml** with the patch template feature:

```
echo '{{ range $name, $spec := .Workloads }}  
{{ range $cname, $_ := $spec.containers }}  
- op: set  
  path: services.{{ $name }}-{{ $cname }}.read_only  
  value: true  
- op: set  
  path: services.{{ $name }}-{{ $cname }}.user
```

```
value: "65532"
- op: set
  path: services.{{ $name }}-{{ $cname }}.cap_drop
  value: ["ALL"]
{{ end }}
{{ end }}' | score-compose init --patch-templates -
```

In this example, this inlined snippet patches all the Workloads and sets the fields `read_only`, `user` and `cap_drop`.

```
score-compose generate score.yaml \
  --image scorespec/sample-score-app:latest
```

See the associated new fields and values in the generated [compose.yaml](#) file.

Instead of using the inlined format, you can use `--patch-templates` with local or external files (Https, Git, OCI), like this:

```
score-compose init \
  --patch-templates https://raw.githubusercontent.com/score-spec/community-
  patchers/refs/heads/main/score-compose/unprivileged.tpl
```

--provisioners

Resource provisioners are the way to define how Platform Engineers want to implement actual resource types defined in the Score file by the Developers.

In the example used so far, see this [score.yaml](#) file, the resource types requested are `postgres`, `dns` and `route`.

The `score-compose init` downloads the default pre-defined resource types provisioners.

See the list of available resource types:

```
score-compose provisioners list -f json | jq -r .[].Type
```

See the actual definitions of these resource provisioners here: [zz-default.provisioners.yaml](#).

This is a best practice to not use the default pre-defined resource types provisioners and bring your own definitions. You can skip the download of the default provisioners by using the `--no-default-provisioners` option:

```
rm -rf .score-compose
score-compose init --no-default-provisioners
```

From this, if you try to generate the `compose.yaml` file from the `score.yaml` file, you will get the errors mentioning that the resource types don't have any resource provisioners defined:

```
score-compose generate score.yaml \
  --image scorespec/sample-score-app:latest
```

```
Error: failed to provision: resource 'dns.default#hello-world.dns' is not supported by any provisioner
```

Bring your own `dns` resource provisioner:

```
score-compose init --no-default-provisioners --provisioners=- <<EOF
- uri: template://custom-dns
  type: dns
  description: Outputs a localhost domain as the hostname.
  outputs: |
    host: localhost
  expected_outputs:
    - host
EOF
```

Regenerate the `compose.yaml` file accordingly and see that the `dns` provisioner error is gone, the next error will be about `postgres`:

```
score-compose generate score.yaml \
  --image scorespec/sample-score-app:latest
```

```
Error: failed to provision: resource 'postgres.default#hello-world.db' is not supported by any provisioner
```

Validate the resource types supported at this stage:

```
score-compose provisioners list
```

Bring back the default resource provisioners to fix the `postgres` and `route` resource types errors as well as bringing our own `dns` resource provisioner to override the default one:

```
rm -rf .score-compose
score-compose init --provisioners=- <<EOF
- uri: template://custom-dns
  type: dns
  description: Outputs a localhost domain as the hostname.
  outputs: |
    host: localhost
  expected_outputs:
    - host
EOF
```

Validate the resource types supported now:

```
score-compose provisioners list
```

Regenerate the `compose.yaml` file accordingly and see that we don't any error anymore:

```
score-compose generate score.yaml \
  --image scorespec/sample-score-app:latest
```

See the actual `outputs` generated by the associated resource types used:

```
score-compose resources list
```

UID	OUTPUTS
dns.default#hello-world.dns	host
postgres.default#hello-world.db	database, host, name, password, port, username
route.default#hello-world.route	

Resources

- [score-compose CLI](#)
- [Patch templates](#)
- [Patch templates examples Hub](#)

- [Resource provisioners](#)
- [Resource provisioners examples Hub](#)

Advanced scenario with **score-k8s**

The **score-k8s** implementation CLI provides more options to enrich the default **manifests.yaml** file generated in order to support more native Kubernetes features.

--format yaml|kyaml

Export the **manifests.yaml** as [kyaml](#) instead of default **yaml**:

```
score-k8s generate score.yaml \  
  --image scorespec/sample-score-app:latest \  
  --format kyaml
```

See the new format of the generated [manifests.yaml](#) file.

--override-property

Override any properties of the Score file imperatively:

```
score-k8s generate score.yaml \  
  --image scorespec/sample-score-app:latest \  
  --override-property containers.hello-world.variables.MESSAGE="Hello, Kubernetes!"
```

See the new **MESSAGE** value in the generated [manifests.yaml](#) file.

--namespace

Set the **namespace** field for the generated Kubernetes resources:

```
score-k8s generate score.yaml \  
  --image scorespec/sample-score-app:latest \  
  --namespace test
```

See the new **namespace** field in the generated [manifests.yaml](#) file.

--generate-namespace

Generate a **Namespace** Kubernetes resource:

```
score-k8s generate score.yaml \  
  --image scorespec/sample-score-app:latest \  
  --namespace test \  
  --generate-namespace
```

See the new **Namespace** resource in the generated [manifests.yaml](#) file.

--patch-templates

Override the default content and any fields of the **manifests.yaml** with the patch template feature:

```
echo '{{ range $i, $m := .Manifests }}  
{{ if eq $m.kind "Deployment" }}  
- op: set  
  path: {{ $i }}.spec.template.spec.automountServiceAccountToken  
  value: false  
- op: set  
  path: {{ $i }}.spec.template.spec.securityContext  
  value:  
    fsGroup: 65532  
    runAsGroup: 65532  
    runAsNonRoot: true  
    runAsUser: 65532  
    seccompProfile:  
      type: "RuntimeDefault"  
{{ range $cname, $_ := $m.spec.template.spec.containers }}  
- op: set  
  path: {{ $i }}.spec.template.spec.containers.{{ $cname }}.securityContext  
  value:  
    allowPrivilegeEscalation: false  
    privileged: false  
    readOnlyRootFilesystem: true  
    capabilities:  
      drop:  
        - ALL  
{{ end }}  
{{ end }}  
{{ end }}' | score-k8s init --patch-templates -
```

In this example, this inlined snippet patches all the Workloads and sets the **securityContext** fields.

```
score-k8s generate score.yaml \  
  --image scorespec/sample-score-app:latest
```

See the associated new fields and values in the generated [manifests.yaml](#) file.

Instead of using the inlined format, you can use `--patch-templates` with local or external files (Https, Git, OCI), like this:

```
score-k8s init \  
  --patch-templates https://raw.githubusercontent.com/score-spec/community-patchers/refs/heads/main/score-k8s/unprivileged.tpl \  
  --patch-templates https://raw.githubusercontent.com/score-spec/community-patchers/refs/heads/main/score-k8s/service-account.tpl
```

In this example above, in addition to set the `securityContext` with this [unprivileged.tpl](#) patch template file, we are also using a second [service-account.tpl](#) patch template file to create a dedicated `ServiceAccount` resource per `Deployment`.

Provisioners

Resource provisioners are the way to define how Platform Engineers want to implement actual resource types defined in the Score file by the Developers.

In the example used so far, see this [score.yaml](#) file, the resource types requested are `postgres`, `dns` and `route`.

The `score-k8s init` downloads the default pre-defined resource types provisioners.

See the list of available resource types:

```
score-k8s provisioners list -f json | jq -r .[].Type
```

See the actual definitions of these resource provisioners here: [zz-default.provisioners.yaml](#).

This is a best practice to not use the default pre-defined resource types provisioners and bring your own definitions. You can skip the download of the default provisioners by using the `--no-default-provisioners` option:

```
rm -rf .score-k8s  
score-k8s init --no-default-provisioners
```

From this, if you try to generate the `manifests.yaml` file from the `score.yaml` file, you will get the errors mentioning that the resource types don't have any resource provisioners defined:

```
score-k8s generate score.yaml \  
  --image scorespec/sample-score-app:latest
```



```
Error: failed to provision: resource 'dns.default#hello-world.dns' is not supported by any provisioner
```

Bring your own **dns** resource provisioner:

```
score-k8s init --no-default-provisioners --provisioners=- <<EOF
- uri: template://custom-dns
  type: dns
  description: Outputs a localhost domain as the hostname.
  outputs: |
    host: localhost
  expected_outputs:
    - host
EOF
```

Regenerate the **manifests.yaml** file accordingly and see that the **dns** provisioner error is gone, the next error will be about **postgres**:

```
score-k8s generate score.yaml \
  --image scorespec/sample-score-app:latest
```

```
Error: failed to provision: resource 'postgres.default#hello-world.db' is not supported by any provisioner
```

Validate the resource types supported at this stage:

```
score-k8s provisioners list
```

Bring back the default resource provisioners to fix the **postgres** and **route** resource types errors as well as bringing our own **dns** resource provisioner to override the default one:

```
rm -rf .score-k8s
score-k8s init --provisioners=- <<EOF
- uri: template://custom-dns
  type: dns
  description: Outputs a localhost domain as the hostname.
  outputs: |
    host: localhost
  expected_outputs:
    - host
EOF
```

Validate the resource types supported now:

```
score-k8s provisioners list
```

Regenerate the `manifests.yaml` file accordingly and see that we don't any error anymore:

```
score-k8s generate score.yaml \  
  --image scorespec/sample-score-app:latest
```

See the actual `outputs` generated by the associated resource types used:

```
score-k8s resources list
```

UID	OUTPUTS
dns.default#hello-world.dns	host
postgres.default#hello-world.db	database, host, name, password, port, username
route.default#hello-world.route	

Resources

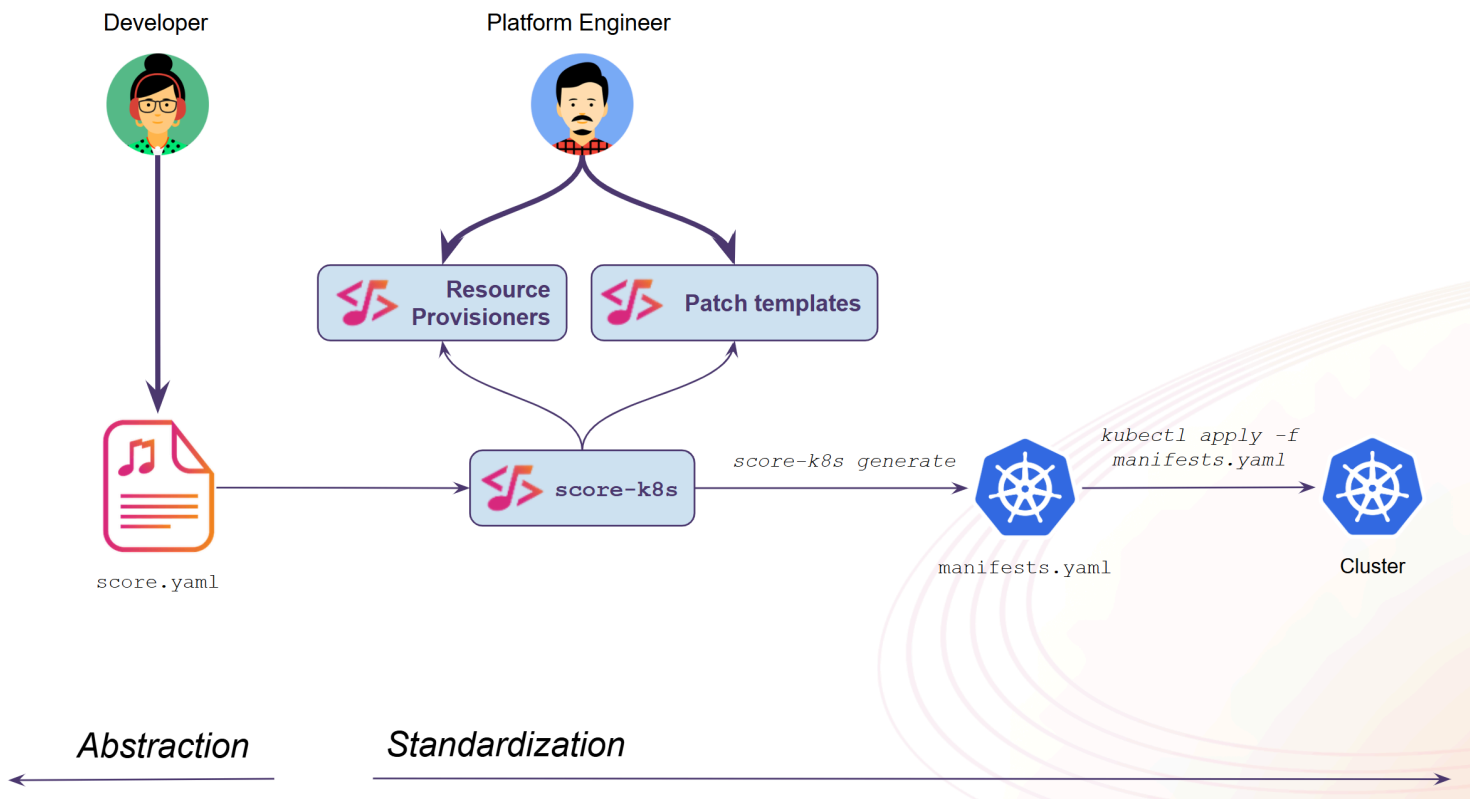
- [score-k8s CLI](#)
- [Patch templates](#)
- [Patch templates examples Hub](#)
- [Resource provisioners](#)
- [Resource provisioners examples Hub](#)

Conclusion

You've completed the **Score Labspace**! 🎉

✅ You now know:

- What is Score (Spec versus Implementations)
- How to author a Score file as Developers
- How to use the two default Score implementations: `score-compose` and `score-k8s`
- How to customize the outputs of the Score implementations with the patch templates feature
- How to define your own resource types with the resource provisioners feature



🎉 Well done!

Next Steps

- [Explore more examples](#)
- [CI/CD pipelines with Score](#)
- [Join the Score community](#)
- Any suggestions to improve this **Score Labspace**? Please help us make it better by filing an issue [there](#)!

Let's Score it! 🎵